Lösningsförslag till gamla tentor i TDDD20 - Konstruktion och Analys av Algoritmer - Från Linköpings Universitet.

Förord

Följande dokument innehåller några utav mina lösningar till problem från gamla tentor i TDDD20. Notera följande:

- Inga beskrivningar utav själva uppgiften förekommer i detta dokument utan måste hämtas av läsaren själv.
- Notera att lösningarna endast är rubricerade under det datum på tentan som uppgiften kommer ifrån.
- Då det alltid finns flera sätt att lösa ett problem, ska dessa lösningar endast ses som lösningsförslag.
- Det kan vara svårt att ”rätta” en algoritm eller ett bevis. Således kan det finnas mindre fel som indexeringsfel eller ”slarviga” argument och resonemang.
- Lösningarna försöker följa en logisk struktur där man i ordning presenterar: algoritmen, tidskomplexiteten och korrektheten.

Innehåll

2023-06-07 2
Uppgift 2. (8p) 2
Uppgift 3. (8p) 3
Uppgift 4. (8p) 4
2024-03-22 5
Uppgift 2. (8p) 5
2023-08-22 6
Uppgift 4. (8p) 6
2023-03-24 7
Uppgift 1. (8p) 7
Uppgift 2. (8p) 8
1. 8
2. Reduktion från NAE-3SAT: 8
Uppgift 3. (8p) 9
Uppgift 4. (8p) 10
Uppgift 5. (8p) 11

2023-06-07

Uppgift 2. (8p)

För att visa att problemet är *NP-svårt* måste vi skapa en korrekt reduktion, som går i polynomisk tid, från ett känt NP-svårt problem till det nya QUADRATIC-EQS.

Reduktion från 3SAT

Notera:

1. För en godtycklig klausul, $(x \vee y \vee \neg z)$, kan vi skriva om den som en ekvation, där u, v är slackvariabler:

$$(x \vee y \vee \neg z) \underset{u,v}{\rightarrow} x + y + (1 - z) - u - v = 1 \mid x, y, z, u, v \in \{0, 1\}$$

2. Följande ekvationssystem tvingar, om $p, q \in \mathbb{Q}$, att $p, q \in \{0, 1\}$:

$$\begin{cases} p^2 + q = 1 \\ q^2 + p = 1 \end{cases} \mid p, q \in \mathbb{Q} \Rightarrow \text{Lösningar: } \begin{cases} p = 0, \ \ q = 1 \\ p = 1, \ \ q = 0 \end{cases}$$

Notera att $\sqrt{5} \notin \mathbb{Q}$ så endast dessa två lösningar gäller. Det gäller också att $p \Leftrightarrow \neg q$ och $q \Leftrightarrow \neg p$.
För varje unik klausul från 3SAT skapar vi ekvationer enligt följande mönster:

$$(x \vee y \vee \neg z) \underset{u,v}{\rightarrow} \begin{cases} x^2 + \hat{x} = 1 \\ \hat{x}^2 + x = 1 \\ y^2 + \hat{y} = 1 \\ \hat{y}^2 + y = 1 \\ z^2 + \hat{z} = 1 \\ \hat{z}^2 + z = 1 \\ u^2 + \hat{u} = 1 \\ \hat{u}^2 + u = 1 \\ v^2 + \hat{v} = 1 \\ \hat{v}^2 + v = 1 \\ x + y + (1 - z) - u - v = 1 \end{cases} \mid x, y, z, u, v \in \mathbb{Q}$$

Följ detta mönster för alla klausuler där negerade literaler $\neg l$ sätts till $(1 - l)$ i ekvationen.

Tidskomplexitet:
Reduktionen lägger för varje klausul till 11 ekvationer. Antag att det går i polynomisk tid att skapa en ekvation. Då går det i $O(|C| \times 11 \times \text{poly})$-tid, där C är mängden klausuler, att skapa hela den nya instansen. Detta är polynomisk tid.

Korrekthet:
Om 3SAT ger "ja" kan man använda samma tilldelning från 3SAT i QUADRATIC-EQS genom att sätta variabler enligt "SANT" $\mapsto 1$ eller "FALSKT" $\mapsto 0$. Enligt notering 1 och 2 följer det då att detta är en korrekt lösning till det stora ekvationssystemet. Alltså måste QUADRATIC-EQS svara "ja".
Om QUADRATIC-EQS ger "ja" har vi alltså enligt notering 2 tilldelat en lösning så att varje variabel $x \mapsto \{0, 1\}$. Notering 1 säger då att, eftersom alla ekvationer är lösta och lösningen på den 11:e ekvationen i varje grupp är ekvivalent med originalklausulen, måste formeln satisfieras av ekvationssystemets lösning. Alltså måste 3SAT svara "ja".

Det följer alltså att QUADRATIC-EQS är NP-svårt eftersom $\text{3SAT} \underset{p}{\leq} \text{QUADRATIC-EQS}$.

Q.E.D

Uppgift 3. (8p)

Skapa listan K där $K[i]$ är 1 om $s[1...i]$ är (K)onstruerbar i X. Vi får då följande rekursiva relation:

$$K[i] = \begin{cases} 1 & \text{om } s[1...i] \in X \\ \bigvee_{j=1}^{i-1} [K[j] \wedge s[j+1...i] \in X] & \text{annars} \end{cases}$$

och kan fylla K med följande algoritm.

Algoritm:

KONSTRUERBAR(s, X)

1 $L := s.\text{length}$
2 $K[0...L] := 0$
3 $K[0] := 1$
4 for i in $[1...L]$:
5 $\quad K[i] := \begin{cases} 1 & \text{om } s[1...i] \in X \\ \bigvee_{j=1}^{i-1} [K[j] \wedge s[j+1...i] \in X] & \text{annars} \end{cases}$
6 return ”ja” om $K[L] = 1$ annars ”nej”.

Tidskomplexitet: Algoritmen fyller i en lista av längd L. För varje element som fylls i behöver max $L - 1$ föregående element kontrolleras. Algoritmen går således i $O(L^2)$-tid där L är längden på strängen s.

Korrekthet: Vi visar korrektheten med ett induktionsbevis över $L := s.\text{length}$.

Basfall: $L = 0$
Då $K[0] := 1$ i steg 3, kommer algoritmen att returnera ”ja”. Detta är korrekt då den tomma strängen alltid är konstruerbar (genom att lägga till 0 tecken).

Hypotes: Antag att algoritmen är korrekt för alla $L \leq k$.
Steg: $L = k + 1$
I steg 4–5 kommer algoritmen att kontrollera alla prefix av s upp till längd $k + 1$. Notera att en sträng s är konstruerbar i X om och endast om det finns en delningspunkt i så att:
$K[i] = 1$ och $s[i + 1...L] \in X$.
Algoritmen letar efter alla sådana delningspunkter. Notera nu också att for-loopen går i stigande ordning så strängar av längd k har redan kontrollerats. Induktionshypotesen kan nu appliceras och det följer att algoritmen är korrekt.

Q.E.D

Uppgift 4. (8p)

1-in-3SAT

Algoritm: För varje variabel sätt den med 50% sannolikhet till 1, annars 0.

Tidskomplexitet: detta går uppenbarligen i O(n) tid över antalet variabler genom att använda en slumpbit.

Korrekthet: Vi får nu åtta möjliga utfall för varje klausul, där 3 av dem är satiriserande för klausulen. Notera alla dessa åtta utfall har samma sannolikhet då vi satte 50/50 över variablerna. För varje klausul skapa en stokastisk variabel X_i som är 1 om klausulen har en och endast en litteral sann annars 0. Väntevärdet för en sådan variabel är nu $E(X_i) = \frac{3}{8}$. Väntevärdet av antalet satisfierade klausuler är nu $E(C) = E(\sum(X_i))$ for i from 1 to $|C|$. Med linearitet är detta samma som $E(C) = \sum(E(X_i))$ för varje X_i. Vilket är detsamma som $|C| \times \frac{3}{8}$. Eftersom att $\frac{3}{8}$ är större än $30\% = \frac{3}{10}$ så är kravet uppfyllt korrekt.

Q.E.D

2024-03-22

Uppgift 2. (8p)

För att visa NP-fullständighet måste vi visa 1. att problemet är i NP och 2. NP-svårhet genom en korrekt reduktion från ett känt NP-svårt problem.

1. Visa att problemet är i NP.
   En ”lösning” till problemet skulle alltså vara två mängder index som tillsammans har längd k. Gå helt enkelt igenom de två mängderna, använd indexen och beräkna summorna för de två mängderna och sedan kollar man att summorna är lika. Detta går i $O(k)$ tid, alltså är problemet i NP.
2. Reduktion från *SUBSETSUM*.
   Om SUBSETSUM ger ”ja” har vi en mängd heltal S och något subset som summerar till T. Vi vill att listan L för det nya problemet ska tvinga fram att T måste vara en delsumma i båda partitionerna. Gör följande:

   $$A := \sum_{a \in S} a$$

   $$L := [S, ..., 2A - T, A + T]$$

   Den totala summan för L är $4A$. Alltså måste en partition exakt ha summan $2A$. Notera att det gäller från SUBSETSUM att $T \geq A$. Det betyder att elementen $2A - T$ och $A + T$ inte kan vara i samma partition. Eftersom: Den partition som väljer elementet $2A - T$ måste då logiskt också välja en delmängd utav S som summerar till exakt T.
   Om SUBSETSUM ger ”ja” ger det nya problemet ”ja”. Eftersom $2A - T$ kommer va i den ena partition. Samma partition måste kan sedan, enligt lösningen som existerar för SUBSETSUM, välja en delmängd utav S som summerar exakt till T. Denna summa är exakt $2A$ och resten av elementen måste nu vara i den andra partitionen. Alltså svarar det nya problemet ”ja”.
   Om det nya problemet svarar ”ja” så måste återigen $2A - T$ vara i den ena partitionen och $A + T$ i den andra eftersom de inte får överskrida $2A$. För att partitionen som innehåller $2A - T$ ska gå att summera upp till exakt $2A$ måste det alltså finnas en delmängd utav S som summerar till exakt T som inkluderats i denna partition. Detta betyder att SUBSETSUM måste svara ”ja”.

   Det nya problemet är NP-fullständigt.

Q.E.D

2023-08-22

Uppgift 4. (8p)

För att visa att Horn+SAT är NP-fullständigt måste vi visa två saker: att problemet tillhör komplexitetsklassen NP, samt att det är NP-svårt genom en reduktion från ett känt NP-svårt problem (i detta fall 3SAT).

1. NP-medlemskap:
Horn+SAT är i NP precis som SAT och 3SAT och en lösning kan alltså verifieras i polynomisk tid.

2. Reduktion från 3SAT:
Vi visar NP-svårighet genom reduktion från 3SAT. För godtyckliga 3SAT-klausuler finns det endast två typer som inte uppfyller kraven för Horn+. Dessa är av formerna:

$$(x \vee y \vee z) \quad \text{och} \quad (x \vee y \vee \neg z)$$

(Notera att inbördes ordning på literalerna kan ändras)

För varje 3SAT-klausul som bryter mot Horn+-kraven byter vi ut den mot en ekvisatisfierbar mängd nya klausuler. För varje utbyte inför vi 2 eller 3 nya *unika* variabler för den nya instansen:

$$(x \vee y \vee z) \underset{p,q,w}{\rightarrow} (x \vee p) \wedge (y \vee q) \wedge (z \vee w) \wedge (\neg p \vee \neg q \vee \neg w)$$

$$(x \vee y \vee \neg z) \underset{p,q}{\rightarrow} (x \vee \neg z \vee \neg p) \wedge (y \vee \neg z \vee \neg q) \wedge (p \vee q)$$

Metod: Notera att man kan komma framm till denna reduktion genom att försöka ”skriva av” 3SAT formeln till Horn+SAT och göra en ersättning när man skulle brutit mot Horn+SAT kraven. Exempelvis för $(x \vee y \vee z)$ börjar man skriva $(x \vee y)$ eftersom att denna klausulen är godkänd. Man noterar sedan att om endast z från 3SAT är satisfierad funkar inte klausulen. Alltså måste man sätta in en hjälp variabel. Man får då $(x \vee p) \wedge (y \vee q) \wedge (z \vee w)$. Därifrån noterar man att om $x, y, z \mapsto 0$ är formeln fortfarande satisfierbar genom $p, q, w \mapsto 1$ och man måste motverka detta genom klausulen $(\neg p \vee \neg q \vee \neg w)$, som säger att alla tre hjälp variabler inte får vara sanna samtidigt. Alltså måste minst en av x, y, z vara sann.

Tidskomplexitet:
Reduktionen går uppenbart i polynomisk tid. För varje klausul i originalformeln (om den inte redan uppfyller kraven) inför vi maximalt 4 nya klausuler och högst 3 nya variabler. Detta innebär en linjär expansion, vilket går i $O(|C|)$-tid, där C är mängden klausuler i originalinstansen.

Korrekthet:
Eftersom vi för varje 3SAT-klausul som inte uppfyller Horn+-kraven skapar en mängd nya klausuler som är lokalt ekvisatisfierbara, kommer hela 3SAT-instansen och den nya Horn+-instansen att vara globalt ekvisatisfierbara.

- **Om 3SAT svarar ”ja”**: Samma variabeltilldelning gör den nya Horn+-formeln satisfierad, förutsatt att vi tilldelar de nya variablerna p, q, w korrekta sanningvärden. Alltså svarar Horn+SAT ”ja”.
- **Om Horn+SAT svarar ”ja”**: Då kan 3SAT också svara ”ja” genom att ta tilldelningen från lösningen för Horn+SAT och helt enkelt bortse från de godtyckligt införda hjälpvariablerna p, q och w. De ursprungliga literalerna måste vara tilldelade på ett sätt som satisfierar originalklausulerna.

Således är reduktionen korrekt. Slutsatsen är att Horn+SAT är NP-fullständigt.

Q.E.D

2023-03-24

Uppgift 1. (8p)

Algoritm:

Skapa tabellen L fylld med 0 som är $(K+1) \times (X+1)$ stor. $L[i][j]$ representerar antalet sätt att skapa summan j med i tal ur $\{1, ..., s\}$.

NR-SOLUTIONS(k, s, X)

```
L[0][0] := 1 // Basfall: 1 sätt att få summan 0 med 0 element.
för i från 1 till k:
    S := 0 // glidande fönster
    för j från 1 till X:
        om j − 1 ≥ 0:
            S += L[i − 1][j − 1] // Lägg till nytt element i fönstret.
        om j − 1 − s ≥ 0:
            S −= L[i − 1][j − 1 − s] // Ta bort gammalt element ur fönstret.
        L[i][j] := S
Returnera L[k][X]
```

Tidskomplexitet: Det framgår från algoritmens for-loopar att den går i $O(k \cdot X)$-tid.

Korrekthet: Bevisas med induktion över antalet variabler i.

Basfall: $i = 0$. Algoritmen sätter $L[0][0] = 1$ och $L[0][j] = 0$ för $j > 0$. Detta är korrekt då det enda sättet att bygga en summa med noll variabler är om summan är precis noll.

Hypotes: Antag att algoritmen korrekt beräknar $L[i-1][j]$ för alla summor $j \in \{0, ..., X\}$.

Steg: För att skapa summan j med i stycken variabler $x_1, ..., x_i$, vet vi att x_i kan anta vilket värde v som helst mellan 1 och s. De resterande $i-1$ variablerna måste då summera till $j - v$. Det totala antalet sätt blir då:

$$L[i][j] = \sum_{v=1}^{s} L[i-1][j-v]$$

I algoritmen uppdateras variabeln S för varje j genom att vi adderar $L[i-1][j-1]$ och subtraherar det element som ligger s steg bort, $L[i-1][j-1-s]$. Notera alltså att $L[i-1][j-1-s]$ hade ”krävt ett tal v sådant att $j - 1 - s + v = X \Rightarrow v \geq s + 1$ vilket inte går. Detta innebär att S exakt håller summan av de senast passerade elementen i intervallet $[j-s, j-1]$ från föregående rad. Matematisk motsvarar detta exakt summan ovan. Hypotesen kan nu appliceras och alla $L[i-1]$ är korrekta, vilket medför att algoritmen korrekt beräknar $L[i][j]$.

Således är algoritmen korrekt.

Q.E.D

Uppgift 2. (8p)

Vi visar 1. att problemet är i NP och 2. att problemet är NP-svårt.

1.

Problemet är i NP då en lösning kan verifieras i $O(4 \times |C|)$-tid där C är mängden klausuler. Detta sker genom att kontrollera de 4 literalerna i varje klausul.

2. Reduktion från NAE-3SAT:

NAE-3SAT är ett känt NP-fullständigt problem där varje klausul har exakt 3 litteraler och ska tilldelas så att minst 1 litteral är sann och 1 litteral är falsk i varje klausul.

Notera detta kan verifieras genom Google.

Notera nu att ingen reduktion krävs eftersom att varje klausul har exakt 3 litteraler. Alltså har alla klausuler högst 4 litteraler.

Korrekthet: följer direkt ifrån NAE-3SAT.

Q.E.D

Uppgift 3. (8p)

Algoritm:

```
MULTIPLE-MST
T = (V_T, E_T) := är ett MST från Prim/Kruskals algoritm.
Ω := Totala vikten av T.
för e i E_T:
    E' := E \ {e}
    G' := grafen av (V, E')
    S := är G' sammanhängande? // Detta kollas med BFS/DFS.
    om inte S:
        Nästa iteration
    A := MST av G'
    Ω_A := Vikten av A
    om Ω_A = Ω:
        Returnera "ja"
Returnera "nej"
```

Tidskomplexitet: Notera att: MST kan beräknas i $O(|E|\log|V|)$. BFS, DFS samt totala vikten av en graf kan beräknas i $O(|V|+|E|)$. For-loopen går över varje $e \in E_T$ och totalt blir det:

$$O(|E_T| \times |E|\log|V|)$$

Detta är polynomisk tid, vilket krävdes.

Korrekthet: Algoritmen testar uteslutande alla MST av G'. Det nya MST A måste vara $A \neq T$ eftersom G' tagit bort en kant från E_T. Algoritmen är således korrekt genom uteslutning.

Q.E.D

Uppgift 4. (8p)

Algoritm:

INTERVAL-COVER(I)

```
n := |I|
if n ≤ 0: return 0
Sortera I på a_i för varje [a_i, b_i], i stigande ordning.
S := 0;
a := I[1].a
b := I[1].b
för varje par [a_i, b_i] och [a_{i+1}, b_{i+1}] i I:
    if a ≤ a_{i+1} ≤ b: // överlappar!
        b := max(b, b_{i+1})
    else:
        S += b − a
        a := a_{i+1}
        b := b_{i+1}
S += b − a
Returnera S
```

Tidskomplexitet: Mergesort som går i $O(n \log n)$ kan användas för sorteringen. Det framgår då från algoritmen att den är $O(n \log n + n \times c)$ där c är en konstant. Detta förenklas till $O(n \log n)$.

Korrekthet: Induktion över antalet intervall n.

Basfall: $n = 1$. Vi går inte in i for-loopen men $S := b_1 - a_1$ beräknas och returneras korrekt.

Hypotes: Antag att algoritmen korrekt beräknar S, a, b för $n \leq k$.

Steg: Beräkningen av intervall $k + 1$ kommer alltså ske i det sista steget utanför for-loopen. Om de sista θ intervallen överlappar kommer $b := b_{k+1}$ och $a := a_{k+2-\theta}$ i if-satsen eftersom $a \leq a_{k+1} \leq b$ måste vara sannt på grund av den sorterade ordningen av intervallen. Summan inkrementeras med $b_{k+1} - a_{k-\theta}$ från att ha varit korrekt uträknad för de $k + 2 - \theta$ föregående intervallen enligt hypotesen. Notera att $\theta \geq 2$.

Om de två sista intervallen är disjunkta kommer summan S vara korrekt för de k föregående intervallen enlight hypotsen och eftersom algoritmen märker att de två sista intervallen är disjunkta i else-satsen på grund av den sorterade ordningen och faktumet att $a_{k+1} \geq b_k$ för att inte överlappa. Nu kommer $a := a_{k+1}$ och $b := b_{k+1}$ i samma else-sats. Sedan inkrementeras S korrekt med $b_{k+1} - a_{k+1}$ i sista steget.

Q.E.D

Uppgift 5. (8p)

Algoritm: För varje nod V tilldelar vi den uniformt, alltså med $\frac{1}{3}$ chans, heltalet 1, 2 eller 3. Detta representerar om noden är i V_1, V_2 eller V_3.

Tidskomplexitet:

Man kan göra varje tilldelning i konstant tid genom rejection sampling, bokens permutationsalgoritm eller en 6-sidig tärning. Alltså går algoritmen i $O(|V|)$-tid vilket är polynomiskt.

Korrekthet: För varje kant $(u, v) \in E$ som har tilldelats $u \mapsto V_x$ och $v \mapsto V_y$ finns följande fall enligt mönstret:

$$u \mapsto 1 \leftrightarrow v \mapsto 1 \qquad u \mapsto 1 \leftrightarrow v \mapsto 2 \qquad u \mapsto 1 \leftrightarrow v \mapsto 3$$

Notera att $\frac{2}{3}$ av dessa är ”gynnsamma”. Låt X_e vara 1 om kant e uppfyller kravet, annars 0. Väntevärdet av algoritmen är nu:

$$E\Big(X_1 + ... + X_{|E|}\Big) \Rightarrow \text{Linjäritet} \Rightarrow E(X_1) + ... + E\Big(X_{|E|}\Big) = |E| \times \frac{2}{3} \times 1 = \frac{2}{3} \times |E|$$

Notera att $\frac{2}{3}\ |E| \geq \frac{2}{3}K$ omm $K \leq |E|$, vilket är logiskt.

Q.E.D